

AWS Secure Landing Zone — Analyse technique détaillée

Terraform + EKS durci : justification des choix de conception, revue module par module, et notes de préparation aux entretiens

Préparé pour Israel Krizoua
2026-07-13

1. Résumé exécutif
2. Pourquoi ce projet, et pourquoi ce design
3. Modèle de menace (ce contre quoi ce design protège)
4. Schémas d'architecture
 - 4.1 Topologie réseau et Kubernetes
 - 4.2 Flux de confiance CI/CD et accès break-glass
5. Analyse détaillée des modules Terraform
 - 5.1 `terraform/modules/vpc`
 - 5.2 `terraform/modules/kms`
 - 5.3 `terraform/modules/iam`
 - 5.4 `terraform/modules/eks`
 - 5.5 `terraform/environments/prod`
6. Analyse détaillée de la couche de sécurité Kubernetes
 - 6.1 Pod Security Standards (`kubernetes/namespace.yaml`)
 - 6.2 NetworkPolicies (`kubernetes/network-policies/`)
 - 6.3 RBAC (`kubernetes/rbac/`)
 - 6.4 Secrets : External Secrets Operator + IRSA (`kubernetes/external-secrets/`)
 - 6.5 Exemple de Deployment durci (`kubernetes/pod-security/hardened-deployment-example.yaml`)
7. Analyse détaillée du pipeline CI/CD
 - 7.1 `.github/workflows/terraform-security-scan.yml`
 - 7.2 `.github/workflows/terraform-plan.yml`
 - 7.3 `.github/workflows/image-scan-trivy.yml`
8. Méthodologie de scan et interprétation des résultats
9. Runbook de déploiement (ce qu'appliquer réellement ceci impliquerait)
10. Limites connues et ce qu'une mise en production ajouterait
11. Questions d'entretien anticipées

12. Glossaire

1. Résumé exécutif

Ce document est une référence technique complète pour le projet **AWS Secure Landing Zone** (dépôt : `aws-secure-landing-zone`). Il va au-delà du README : il explique **pourquoi** chaque ressource existe, quelle menace ou exigence elle traite, quelles alternatives ont été envisagées, et ce qu'un relecteur ou un recruteur est susceptible de demander à ce sujet.

Le projet livre de l'Infrastructure-as-Code pour une landing zone AWS pensée sécurité d'abord : un VPC segmenté, un modèle IAM en moindre privilège, un chiffrement KMS centralisé, un cluster EKS durci, une couche de sécurité Kubernetes (Pod Security Standards, RBAC, NetworkPolicies, gestion des secrets), et un pipeline CI/CD qui fait échouer une pull request lorsqu'elle introduit une vulnérabilité. Chaque affirmation de ce document ("0 finding CRITICAL", "aucune action IAM en wildcard", etc.) est étayée par une commande que vous pouvez relancer vous-même — rien n'est avancé sans preuve.

Ce que ce projet démontre à un employeur : la capacité à concevoir une infrastructure cloud avec des contrôles de sécurité intégrés dès le départ (pas ajoutés après un audit), à raisonner sur le modèle de permissions IAM d'AWS avec assez de précision pour écrire des politiques sans wildcard inutile, à durcir un cluster Kubernetes sur toutes ses couches (réseau, identité, admission, secrets), et à intégrer le scan de sécurité dans la CI/CD de façon à ce qu'il bloque réellement les mauvais changements plutôt que de simplement les signaler.

Ce que ce projet ne fait pas : il n'a pas été déployé sur un compte AWS réel. Chaque module Terraform passe `terraform validate` et est scanné avec les mêmes outils que ceux utilisés par le pipeline CI (`tfsec` , `checkov`), mais aucun `terraform apply` n'a été exécuté. C'est une décision de périmètre délibérée, pas un oubli — déployer créerait des ressources facturables (NAT Gateways, control plane EKS, clés KMS) sans aucune charge applicative derrière, ce qui est un mauvais usage du budget pour une pièce de portfolio. Si on te demande "tu l'as vraiment lancé ?", la réponse honnête est : "validé et scanné, pas déployé — et voici exactement pourquoi."

2. Pourquoi ce projet, et pourquoi ce design

Les postes cloud security en 2026 (en particulier dans les ESN et grands comptes en France et dans l'UE) attendent de plus en plus des candidats qu'ils montrent, pas qu'ils racontent : un dépôt GitHub avec du vrai Terraform, de vrais résultats de scan, et un README qui explique les compromis l'emporte sur une liste de certifications. Ce projet a été délimité précisément pour toucher l'intersection de trois compétences habituellement enseignées et démontrées séparément :

1. **Infrastructure as Code** (Terraform : modules, state, variables, validation)
2. **Cloud Security Posture Management** (la discipline consistant à concevoir IAM/réseau/chiffrement de sorte qu'une mauvaise configuration soit structurellement

difficile, et à utiliser des outils comme tfsec/Checkov pour rattraper ce qui passe entre les mailles)

3. **Durcissement Kubernetes** (Pod Security Standards, NetworkPolicies, RBAC, gestion des secrets — les parties de Kubernetes qui sont optionnelles par défaut, et donc celles qu'on saute sous la pression d'un délai)

Le principe de conception unificateur dans tout le dépôt est : **chaque contrôle de sécurité a une justification d'une phrase rattachée à une menace précise**, et **chaque exception à une règle de sécurité est documentée en ligne dans le code, pas balayée dans une liste d'exclusion globale**. Ce second point est délibérément sur-conçu par rapport à ce qu'une équipe réelle pressée par le temps ferait probablement — il existe ici parce qu'il rend le *raisonnement* visible, ce qui est précisément l'objectif d'une pièce de portfolio.

3. Modèle de menace (ce contre quoi ce design protège)

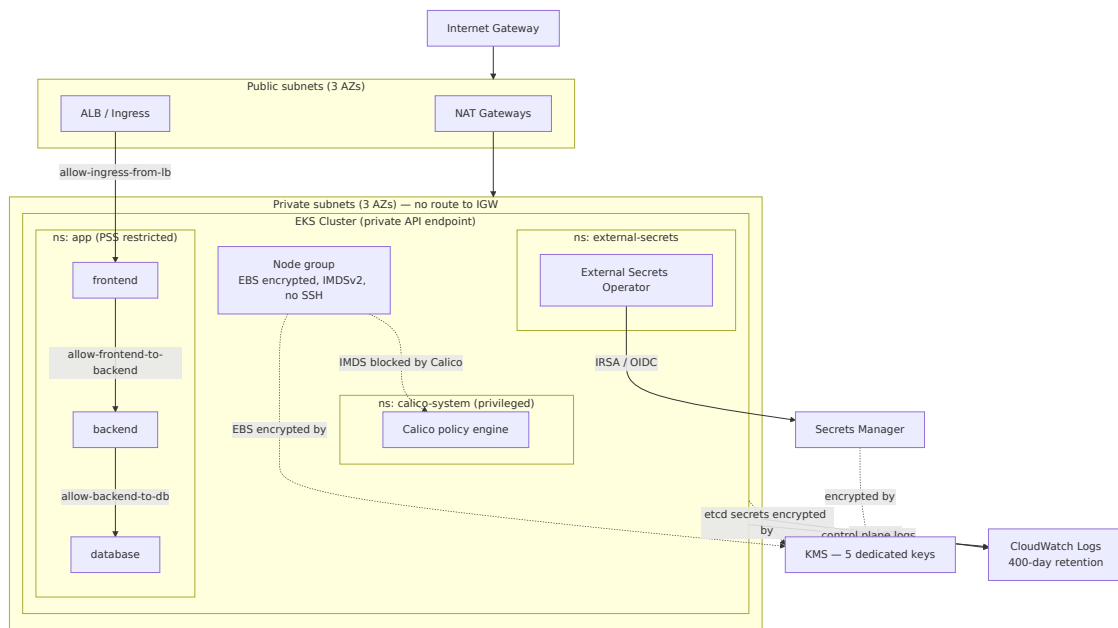
Être explicite sur le modèle de menace est ce qui distingue “j’ai ajouté du chiffrement parce que la checklist le demandait” de “je comprends ce que le chiffrement empêche précisément à cet endroit.” Les menaces autour desquelles cette landing zone est conçue :

Menace	Mitigation principale	Où dans le dépôt
Compromission SSH/RDP d'un nœud exposé sur internet	Aucune route des subnets privés vers l'Internet Gateway ; la NACL refuse explicitement 22/3389 en entrée ; accès aux nœuds via SSM Session Manager uniquement	<code>terraform/modules/vpc/main.tf</code> , <code>terraform/modules/eks/main.tf</code>
Pipeline CI/CD compromis utilisé pour pivoter vers un accès admin complet AWS	<code>iam:PassRole</code> scopé à exactement deux ARN de rôle avec une condition <code>PassedToService</code> ; deny explicite sur la modification de sa propre policy ou la suppression des clés KMS	<code>terraform/modules/iam/main.tf</code> (policy <code>github_actions_deploy</code>)
Fuite de credentials AWS statiques depuis les secrets GitHub Actions	Aucune clé AWS statique du tout — fédération OIDC, session STS de courte durée, confiance scopée à un repo+branche précis	<code>terraform/modules/iam/main.tf</code> (<code>aws_iam_openid_connect_provider.github_actions</code>)
Un pod compromis utilisant du SSRF pour atteindre le service de métadonnées EC2 et voler les	<code>GlobalNetworkPolicy</code> Calico bloquant <code>169.254.169.254</code> depuis tous les pods, quel que soit le namespace,	<code>kubernetes/network-policies/calico-deny-imsd.yaml</code>

Menace	Mitigation principale	Où dans le dépôt
credentials du nœud/de l'instance	indépendamment des NetworkPolicies au niveau namespace	
Un pod compromis se déplaçant latéralement vers un autre tier (ex. frontend → base de données)	<code>NetworkPolicy</code> deny-by-default dans le namespace <code>app</code> ; seuls frontend→backend et backend→database sont explicitement autorisés	<code>kubernetes/network-policies/</code>
Le contexte kubectl d'un développeur touchant accidentellement des ressources cluster-wide	<code>Role</code> / <code>RoleBinding</code> scopés au namespace uniquement — aucun <code>ClusterRole</code> distribué aux équipes applicatives	<code>kubernetes/rbac/</code>
Secrets commités dans Git, ou stockés en clair dans un manifest Kubernetes	L'External Secrets Operator récupère depuis AWS Secrets Manager à l'exécution via IRSA ; aucun objet <code>Secret</code> n'est jamais écrit à la main avec une valeur dedans	<code>kubernetes/external-secrets/</code>
Un attaquant compromettant la session du poste d'un opérateur humain via un credential obsolète/en cache	Le rôle break-glass admin exige <code>aws:MultiFactorAuthPresent=true</code> ET <code>aws:MultiFactorAuthAge < 3600</code> dans la trust policy	<code>terraform/modules/iam/main.tf (break_glass_admin)</code>
Images de base vulnérables ou CVE livrées en production	Trivy scanne les images en CI et fait échouer le build sur CRITICAL/HIGH avec un correctif disponible	<code>.github/workflows/image-scan-trivy.yml</code>
Dérive d'infrastructure réintroduisant une vulnérabilité corrigée	tfsec + Checkov s'exécutent sur chaque PR touchant <code>terraform/</code> , check requis dans la protection de branche	<code>.github/workflows/terraform-security-scan.yml</code>
Investigation post-incident sans piste d'audit	VPC Flow Logs + logs d'audit du control plane EKS, tous deux chiffrés KMS, conservés 400 jours	<code>terraform/modules/vpc/main.tf</code> , <code>terraform/modules/eks/main.tf</code>

4. Schémas d'architecture

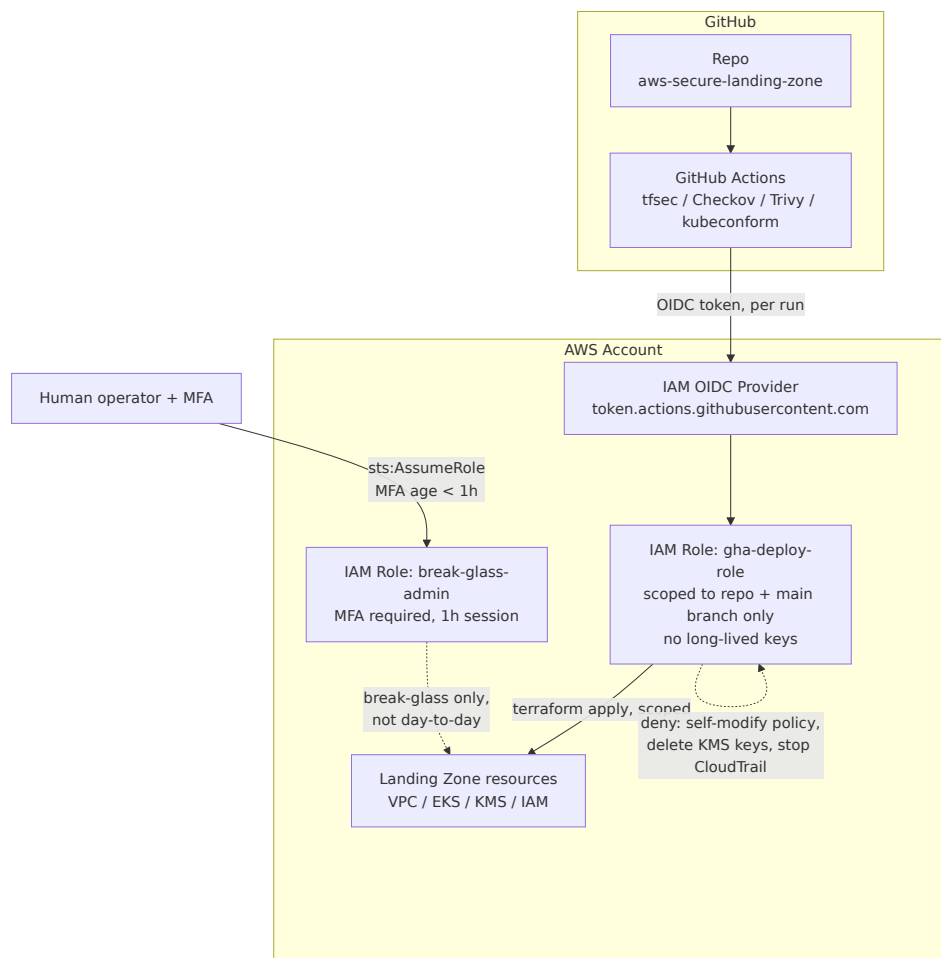
4.1 Topologie réseau et Kubernetes



Topologie réseau et Kubernetes

À l'intérieur du VPC, le seul chemin depuis internet vers une charge de travail est Internet Gateway → subnet public (NAT/ALB) → subnet privé (nœuds EKS) — il n'existe aucune autre route d'entrée. Les pods sont en plus isolés les uns des autres par des NetworkPolicies au niveau Kubernetes (frontend → backend → database, rien d'autre), en complément des security groups au niveau EC2. Chaque flèche en pointillés qui touche KMS représente une donnée chiffrée au repos avec une clé dédiée à cette catégorie de données, pas une clé unique partagée.

4.2 Flux de confiance CI/CD et accès break-glass



Flux de confiance CI/CD et accès break-glass

GitHub Actions ne détient jamais de credential AWS au repos. Il échange un token OIDC de courte durée — émis à neuf par GitHub à chaque exécution de workflow — contre une session STS scoppée à `gha-deploy-role`, lequel se voit lui-même refuser la capacité de toucher sa propre policy ou de détruire les clés KMS/la piste d’audit. Le chemin d’accès break-glass admin est délibérément séparé et rarement emprunté : un opérateur humain ne peut l’assumer qu’avec une session MFA actuellement valide, jamais depuis un pipeline automatisé.

5. Analyse détaillée des modules Terraform

5.1 `terraform/modules/vpc`

Rôle : segmentation réseau et couche de visibilité du trafic.

Ressources et raisonnement :

- `aws_vpc.this` — un seul VPC par environnement, `10.0.0.0/16` par défaut (65 536 adresses — marge confortable pour plusieurs node groups EKS et de futurs subnets sans avoir à re-découper le CIDR).
- `aws_default_security_group.this` — **le security group par défaut du VPC est explicitement capturé et laissé sans aucune règle.** AWS crée automatiquement un security group par défaut avec une règle d'ingress permissive vers lui-même ; si quoi que ce soit est un jour lancé accidentellement sans security group explicite, cela garantit qu'il n'a *aucun* accès réseau plutôt qu'une confiance implicite. C'est un constat d'audit courant (`CKV2_AWS_12` chez Checkov) et une correction à une seule ressource.
- Les subnets publics (`aws_subnet.public` , un par AZ) n'hébergent **que** des NAT Gateways et des load balancers. `map_public_ip_on_launch = false` — même dans un subnet public, une ENI n'obtient pas d'IP publique par défaut ; elle doit être demandée explicitement (comme l'EIP du NAT Gateway l'est). Cela évite qu'une `aws_instance` lancée par erreur dans ce subnet devienne accessible depuis internet par défaut.
- Les subnets privés (`aws_subnet.private`) hébergent les nœuds EKS et les pods. **Il n'existe aucune route vers l'Internet Gateway depuis ces subnets** — le trafic vers internet doit passer par un NAT Gateway. C'est plus robuste que "un security group bloque l'entrée" car il n'existe *aucun chemin de routage* pour qu'un acteur externe atteigne directement ces subnets, indépendamment de toute mauvaise configuration de security group.
- `aws_nat_gateway` — un par AZ par défaut (`single_nat_gateway = false`), pour la disponibilité : si le NAT Gateway d'une AZ ou son EIP a un problème, les subnets privés des autres AZ ne sont pas affectés. Le compromis est le coût (~3x un NAT Gateway unique) et c'est exposé comme variable précisément pour qu'un déploiement sensible au coût (ex. environnement dev/staging) puisse la basculer.
- `aws_flow_log` + `aws_cloudwatch_log_group.flow_logs` — **VPC Flow Logs**, `traffic_type = "ALL"` (connexions acceptées et refusées, pas seulement acceptées), chiffrés avec la clé KMS `cloudwatch_logs` , conservés 400 jours. C'est la couche de forensic réseau : sans elle, "qui a parlé à qui et quand" est impossible à répondre après coup.
- `aws_network_acl.private` — une couche de défense en profondeur *en plus* des security groups. Les NACL sont sans état (elles ne suivent pas l'état des connexions comme le font les security groups), ce qui explique :
 - Les règles 90/91 **refusent explicitement** l'entrée 22/3389 depuis `0.0.0.0/0` — c'est l'exigence "pas de SSH direct depuis internet" rendue structurelle plutôt que conventionnelle. Même si un security group est assoupli par erreur, cette règle NACL bloque quand même.
 - La règle 100 (entrée) autorise tous les ports depuis le CIDR du VPC uniquement — nécessaire pour le trafic nœud-à-nœud et nœud-vers-control-plane ; le contrôle fin par port est laissé aux security groups.
 - La règle 110 (entrée) autorise TCP 1024-65535 depuis `0.0.0.0/0` — cela semble alarmant hors contexte mais c'est la **règle de trafic retour pour NACL sans état** : comme les NACL ne suivent pas l'état des connexions, la réponse à une connexion *initiée en sortie par ce subnet* (ex. un nœud appelant une API AWS via le NAT

Gateway) revient sur un port éphémère arbitraire depuis une IP source arbitraire, et sans cette règle ce trafic retour serait silencieusement rejeté. Elle ne peut pas servir à *initier* une connexion entrante car rien dans ces subnets n'écoute sur ces ports, et les règles 90/91 (numéro de règle plus bas → évaluées en premier) refusent déjà les deux ports qui comptent (22, 3389).

- La règle 100 (sortie) autorise tout — cohérent avec le fait que tout le trafic sortant passe déjà par un unique NAT Gateway ; énumérer les ports de destination ici n'apporte aucun bénéfice de sécurité et nécessiterait de maintenir à la main une liste de ports pour chaque API AWS et registre de paquets auquel le cluster parle.
- Le rôle IAM `flow_logs` — scopé à `logs:CreateLogStream / PutLogEvents / Describe*` sur **l'ARN de ce seul groupe de logs** (avec le suffixe `:*` documenté par AWS pour les log streams qu'il contient), pas `Resource: "*" .`

Findings tfsec/Checkov acceptés délibérément ici (avec un commentaire de justification en ligne à chaque ressource) : les deux règles NACL “tous les ports” et la règle d'entrée sur ports éphémères, pour les raisons ci-dessus. Rien dans ce module n'accepte de finding concernant une entrée depuis internet vers un port de *charge de travail*.

5.2 `terraform/modules/kms`

Rôle : chiffrement au repos, avec confinement du rayon d'impact.

Décision de conception — 5 clés, pas 1 : `eks_secrets` , `ebs` , `s3` , `secrets_manager` , `cloudwatch_logs` . L'alternative (une clé “landing zone” partagée) est plus simple à gérer mais signifie qu'une seule policy de clé trop permissive compromet toutes les catégories de données à la fois. Séparer par domaine de données signifie, par exemple, qu'un bug dans la policy de la clé CloudWatch Logs n'affecte pas la capacité à déchiffrer les volumes EBS ou les entrées Secrets Manager. `enable_key_rotation = true` sur chaque clé (rotation automatique annuelle, gérée par AWS).

Policy de clé : deux statements — administration par le compte root (bonne pratique AWS : ne jamais laisser une policy IAM être le seul chemin vers une clé, sinon une erreur IAM peut rendre la clé définitivement orpheline sans moyen de la corriger) et une liste nommée d'ARN de rôles administrateurs de clé (le rôle break-glass admin plus tout opérateur humain explicitement configuré). Aucun `Principal: "*" .`

5.3 `terraform/modules/iam`

Rôle : c'est le module qui porte le plus la charge de “prouver qu'on comprend vraiment IAM” dans le dépôt, et celui qu'il vaut la peine de savoir expliquer avant un entretien.

Rôles créés :

1. `eks_cluster` — confiance : `eks.amazonaws.com` uniquement. Permissions : `AmazonEKSClusterPolicy` (l'unique policy gérée par AWS qu'EKS requiert). Aucune policy écrite à la main n'est attachée, car il n'y a rien à ajouter — la policy gérée est déjà minimale pour ce dont le control plane lui-même a besoin.

2. `eks_node` — confiance : `ec2.amazonaws.com` uniquement. Permissions : exactement les trois politiques gérées dont les nœuds worker ont besoin (`AmazonEKSWorkerNodePolicy` , `AmazonEKS_CNI_Policy` , `AmazonEC2ContainerRegistryReadOnly`). Pas de S3, pas de RDS, rien d'autre — l'identité IAM d'un nœud compromis peut rejoindre le cluster, faire fonctionner le VPC CNI, et tirer des images. Rien de plus.
3. `github_actions_deploy` — c'est le plus intéressant. La confiance est fédérée via OIDC (`aws_iam_openid_connect_provider.github_actions` , pointant vers `token.actions.githubusercontent.com`), avec la `Condition` de la trust policy restreignant `sub` à `repo:<org>/<repo>:ref:refs/heads/main` — ce qui signifie qu'un run de workflow sur un fork, ou sur toute branche autre que `main` , ne peut pas assumer ce rôle même avec un token OIDC valide émis par GitHub, car la claim `sub` du token ne correspondra pas. **C'est ce que "aucune clé AWS statique stockée en secret GitHub" signifie en pratique** : le workflow appelle `sts:AssumeRoleWithWebIdentity` avec un token que GitHub lui-même émet à chaque run, et AWS valide ce token par rapport à l'empreinte du fournisseur OIDC avant d'émettre des credentials temporaires.

La policy de permissions attachée à ce rôle est l'endroit où l'affirmation "pas de wildcard" est le plus mise à l'épreuve, et mérite d'être parcourue statement par statement (voir `terraform/modules/iam/main.tf` , ressource `github_actions_deploy`) :

- `ReadOnlyDescribeUnavoidablyUnscoped` — `ec2:Describe*` , `eks:Describe*` , etc. sur `Resource: "*"` . Ce n'**est pas** un raccourci : le modèle IAM d'AWS ne prend pas en charge les permissions au niveau ressource pour la plupart des appels `Describe` / `List` sur ces services (documenté dans le guide utilisateur IAM, référence "Resource-level permissions" par service). Il n'existe aucun ARN qu'on pourrait mettre ici qui serait plus précis tout en fonctionnant.
- `ManageVpcNetworking` — une liste organisée d'environ 30 actions `ec2:Create*` / `Delete*` / `Modify*` / `Authorize*` précises (pas `ec2:*`), toujours sur `Resource: "*"` car les ressources réseau EC2 (VPC, subnets, security groups, NACL) ne prennent pour la plupart pas non plus en charge le scoping par ARN au niveau ressource dans IAM — mais la **liste d'actions elle-même** est énumérée exhaustivement plutôt que mise en wildcard, et restreinte davantage par une condition `aws:RequestedRegion` .
- `ManageEksResources` — scopé à `arn:aws:eks:*<compte>:cluster/<name_prefix>-*` et au pattern d'ARN de nodegroup correspondant. EKS *prend* en charge les ARN au niveau ressource, donc ce statement les utilise.
- `ManageLogGroups` — scopé à `arn:aws:logs:*<compte>:log-group:*<name_prefix>*` .
- `ManageProjectIamResources` — scopé à `arn:aws:iam:*<compte>:role/<name_prefix>-*` et au pattern `policy/` correspondant. Le rôle de déploiement peut créer/attacher/tagger les ressources IAM que ce projet possède, et rien nommé en dehors de ce préfixe.
- `PassLandingZoneRolesToAwsServices` — c'est le statement qui mérite d'être mis en avant en entretien, car c'est celui que la plupart des politiques IAM réelles ratent. `iam:PassRole` est ce qui permet à Terraform de transmettre les ARN des rôles `eks_cluster` / `eks_node` aux services EKS/EC2 lors de la création du cluster et du node group. Un `iam:PassRole: Resource: "*"` trop large est l'un des primitifs d'escalade de

5.4 terraform/modules/eks

Rôle : le control plane Kubernetes et la couche de calcul.

- `cluster_endpoint_public_access = false` **par défaut** — c'est un choix de durcissement délibéré, au-delà de ce que le cahier des charges exigeait littéralement. Le raisonnement : Terraform gère la ressource `aws_eks_cluster` via l'API de control plane classique d'AWS (`eks.<région>.amazonaws.com`), qui est un endpoint différent de l'API server Kubernetes propre au cluster. Donc `terraform plan / apply` fonctionne avec **zéro accès réseau au cluster lui-même**, même avec l'endpoint API K8s entièrement privé. Seules les opérations de niveau `kubectl / helm` (installer Calico, l'External Secrets Operator, appliquer des manifests) nécessitent une accessibilité à l'API K8s, et celles-ci sont documentées comme nécessitant un runner auto-hébergé dans le VPC, un VPN, ou du SSM port-forwarding. Si un accès public est un jour nécessaire pour des raisons opérationnelles, `cluster_endpoint_public_access_cidrs` a un bloc `validation` qui fait échouer `terraform plan` s'il contient `0.0.0.0/0`.
- `encryption_config { resources = ["secrets"] }` sur le cluster — chiffre en enveloppe les objets `Secret` Kubernetes dans etcd avec la clé KMS `eks_secrets`, en plus du chiffrement disque natif d'EKS. Sans cela, quiconque a accès à un snapshot `etcd` (un niveau d'accès bien plus bas que l'API K8s) pourrait lire les valeurs de secrets.
- `enabled_cluster_log_types = ["api", "audit", "authenticator", "controllerManager", "scheduler"]` — les cinq flux de logs, pas seulement `audit`. `authenticator` en particulier est ce qui permet de répondre à “qui s'est authentifié en tant que quelle identité IAM et à quel utilisateur Kubernetes était-ce mappé” après un incident.
- Security groups : les SG du control plane et des nœuds n'autorisent que le strict minimum d'échanges croisés (443 des nœuds vers le control plane, ports éphémères du control plane vers les kubelets des nœuds, nœud-à-nœud). L'egress est `0.0.0.0/0` sur les deux — accepté et documenté en ligne (commentaires `tfsec:ignore / checkov:skip`) car les deux SG sont attachés à des ENI qui vivent exclusivement dans des subnets privés sans route IGW, donc ce trafic passe de toute façon par le NAT, et les plages d'IP des API/registres AWS sont trop larges pour être énumérées en liste blanche statique sans maintenance constante.
- `aws_launch_template.nodes` — `http_tokens = "required"` (**IMDSv2 uniquement**, pas de repli vers l'ancien IMDSv1 non authentifié), `http_put_response_hop_limit = 1` (empêche une charge de travail conteneurisée d'atteindre le service de métadonnées de l'instance via un saut réseau supplémentaire, ce qui compte si le trafic d'un pod pourrait sinon être routé via l'hôte), EBS `encrypted = true` avec la clé KMS dédiée `ebs`, `associate_public_ip_address = false`.
- IRSA (`aws_iam_openid_connect_provider.eks`, empreinte récupérée en direct via `data.tls_certificate`) — le mécanisme qui permet à des pods individuels d'assumer des rôles IAM finement scopés au lieu d'hériter du rôle IAM du nœud. Utilisé concrètement par l'External Secrets Operator (voir §6.4).

5.5 terraform/environments/prod

Assemble les quatre modules, plus une ressource qui ne peut vivre ni dans `iam` ni dans `eks` sans créer une dépendance circulaire entre modules : le rôle IRSA pour l'External Secrets Operator, qui a besoin à la fois de l'ARN du fournisseur OIDC du module EKS et de l'ARN de la policy de permissions du module IAM. Sa trust policy est scopée avec des conditions `StringEquals` sur `sub` (`system:serviceaccount:external-secrets:external-secrets` — la paire exacte namespace:serviceaccount, pas un wildcard) et `aud` (`sts.amazonaws.com`).

6. Analyse détaillée de la couche de sécurité Kubernetes

6.1 Pod Security Standards (`kubernetes/namespaces.yaml`)

Le contrôleur d'admission Pod Security intégré à Kubernetes applique l'un des trois profils intégrés par namespace via des labels — aucun opérateur supplémentaire nécessaire. Les namespaces `app` et `external-secrets` sont labellisés `restricted` (le niveau le plus strict : pas de root, pas d'escalade de privilège, `seccompProfile` obligatoire, toutes les capacités Linux retirées par défaut, pas de `hostPath` / `hostNetwork` / `hostPID`). `enforce`, `audit` et `warn` sont tous réglés sur `restricted` — `enforce` rejette purement et simplement les pods non conformes, `audit` / `warn` rendent les violations visibles dans le log d'audit de l'API server et dans la sortie `kubectl`, même pour des éléments déjà bloqués par le niveau `enforce`, ce qui compte pour déboguer pourquoi un manifest a été rejeté. `calico-system` est délibérément laissé en `privileged` — le CNI a besoin de `hostNetwork` / `NET_ADMIN` pour fonctionner, et c'est documenté comme l'unique exception acceptée plutôt que silencieusement exclue.

6.2 NetworkPolicies (`kubernetes/network-policies/`)

La conception est **deny-by-default, allow par exception** — `default-deny-all.yaml` utilise un `podSelector: {}` vide avec `Ingress` et `Egress` tous deux dans `policyTypes`, ce qui bloque tout le trafic vers et depuis chaque pod du namespace `app`, y compris le trafic intra-namespace et le DNS. Tout le reste du répertoire est une exception étroite et additive :

- `allow-dns-egress.yaml` — sans cela, chaque pod échoue à résoudre *quoi que ce soit*, y compris ses propres dépendances, car le deny-by-default bloque aussi la résolution CoreDNS.
- `allow-frontend-to-backend.yaml` / `allow-backend-to-db.yaml` — segmentation par label de pod (`tier: frontend/backend/database`) au sein du même namespace, pas seulement une isolation au niveau namespace. Un pod frontend compromis n'a aucun chemin réseau vers le tier base de données — pas "a été refusé par une règle qui pourrait théoriquement être mal configurée," mais "il n'existe nulle part de règle accordant ce chemin."

- `allow-ingress-from-lb.yaml` — seul le tier frontend accepte du trafic provenant de l'extérieur du réseau de pods.
- `calico-deny-imds.yaml` — une `GlobalNetworkPolicy` Calico (pas l'API `NetworkPolicy` standard), car l'API standard ne peut sélectionner que des pods/namespaces, pas un CIDR externe arbitraire. Cela bloque tous les pods du cluster, quel que soit le namespace, dans leur accès à `169.254.169.254` (métadonnées d'instance) — fermant le chemin SSRF-vers-vol-de-credentials le plus courant sur EKS, indépendamment de la politique au niveau namespace appliquée ou non à une charge de travail donnée.

6.3 RBAC (`kubernetes/rbac/`)

`Role` / `RoleBinding` uniquement — **aucun `ClusterRole` n'est accordé aux équipes applicatives nulle part dans ce dépôt.** `app-developer` obtient la création/mise à jour sur les Deployments/Services/ConfigMaps et la lecture seule sur les Pods/logs, mais explicitement *pas* `exec` / `attach` / `port-forward` vers des pods en cours d'exécution (livrer des manifests n'est pas la même permission qu'obtenir un shell en production) et explicitement *pas* d'accès aux `secrets` (ceux-ci sont exclusivement gérés par le rôle propre, scopé au namespace, de l'External Secrets Operator). Les `RoleBinding` ciblent des **groupes** mappés IAM (`app-developers` , `app-readonly`) plutôt que des utilisateurs nommés individuellement, donc l'onboarding/offboarding est un changement d'appartenance à un groupe côté AWS, pas une modification de manifest.

6.4 Secrets : External Secrets Operator + IRSA (`kubernetes/external-secrets/`)

La chaîne, de bout en bout : le ServiceAccount `external-secrets` est annoté avec l'ARN du rôle IRSA (`eks.amazonaws.com/role-arn`) créé dans `terraform/environments/prod/main.tf` . La policy IAM de ce rôle autorise exactement `secretsmanager:GetSecretValue / DescribeSecret` sur `arn:aws:secretsmanager:*:<compte>:secret:<name_prefix>/*` — uniquement les secrets de ce projet. Un `ClusterSecretStore` pointe l'opérateur vers AWS Secrets Manager, en s'authentifiant via ce service account (aucune clé AWS statique nulle part dans le cluster). Un objet `ExternalSecret` déclare "le namespace `app` a besoin d'un `Secret` appelé `db-credentials` , sourcé depuis la clé Secrets Manager `slz-prod/app/db-credentials` " — l'opérateur interroge et réconcilie automatiquement l'objet `Secret` réel, y compris lors d'une rotation. **La valeur du secret n'apparaît jamais dans ce dépôt Git, dans aucun manifest, ni dans l'historique shell d'un `kubectl apply` .**

6.5 Exemple de Deployment durci (`kubernetes/pod-security/hardened-deployment-example.yaml`)

Inclus précisément pour que les exigences abstraites des Pod Security Standards aient un exemple concret, copiable-collable : `runAsNonRoot: true` , UID/GID non-root explicites, `seccompProfile: RuntimeDefault` , `allowPrivilegeEscalation: false` , `readOnlyRootFilesystem: true` (avec un `emptyDir` monté sur `/tmp` pour l'espace de travail temporaire dont une racine en lecture seule a quand même besoin), `capabilities.drop: ["ALL"]` ,

`automountServiceAccountToken: false` sur un service account sans annotation de rôle IAM (cette charge de travail précise n'a aucune raison de parler aux API AWS, donc elle n'obtient aucune identité AWS — pas même celle du nœud).

7. Analyse détaillée du pipeline CI/CD

7.1 `.github/workflows/terraform-security-scan.yml`

Déclenché sur toute PR touchant `terraform/`, plus après fusion sur `main`. Quatre jobs : `terraform fmt -check` + `terraform validate` par module/environnement ; `tfsec` (`soft_fail: false` — un finding HIGH/CRITICAL fait échouer le job, pas seulement le signaler) ; `checkov` (même principe, plus un upload SARIF vers l'onglet Security de GitHub afin que les findings apparaissent nativement dans la vue "Security" de la PR, pas seulement dans les logs du workflow) ; `kubeconform` pour la validation de schéma des manifests Kubernetes (`-ignore-missing-schemas` scopé précisément aux deux API de ressources personnalisées utilisées par ce dépôt — Calico et External Secrets — afin que les types Kubernetes natifs restent validés strictement). Comme les commentaires `ignore / skip` en ligne de `tfsec/Checkov` sont respectés par les outils eux-mêmes, une exception documentée ne fait pas échouer le build, mais une exception *non documentée* le fait — c'est ce qui rend "bloque la fusion si une faille est détectée" réellement vrai plutôt qu'aspirationnel.

7.2 `.github/workflows/terraform-plan.yml`

Démontre la fédération OIDC de bout en bout : `aws-actions/configure-aws-credentials` échange le token OIDC du workflow contre une session sous `github_actions_deploy_role_arn` (aucun secret `AWS_ACCESS_KEY_ID` nulle part dans les réglages du dépôt), puis exécute `terraform plan` sur l'environnement `prod`. Délibérément `plan-only` — l'apply est traité comme une étape séparée, approuvée manuellement, pas quelque chose qu'une PR déclenche automatiquement.

7.3 `.github/workflows/image-scan-trivy.yml`

Construit l'image applicative (une fois qu'il en existe une dans ce dépôt — le workflow est déclenché par chemin sur `Dockerfile / src/**`, donc il est inerte tant qu'aucun code applicatif n'est ajouté) et la scanne avec Trivy, `exit-code: 1` sur CRITICAL/HIGH avec un correctif disponible (`ignore-unfixed: true` — inutile de bloquer une fusion pour une CVE sans patch applicable). Un second scan non bloquant upload un rapport SARIF pour l'onglet Security, et une étape séparée scanne les manifests `kubernetes/` eux-mêmes à la recherche de mauvaises configurations avec le scanner de config de Trivy.

8. Méthodologie de scan et interprétation des résultats

Deux scans ont été exécutés pour la comparaison “avant/après” demandée dans le cahier des charges, en utilisant exactement les versions d’outils que le pipeline CI utilise (`tfsec v1.28.13` , `checkov v3.3.8`) :

- **Avant** : `examples/insecure-baseline/main.tf` — un fichier Terraform délibérément cassé (SSH/RDP ouverts à `0.0.0.0/0` , un bucket S3 public, un volume EBS non chiffré, une policy IAM avec `Action: "*", Resource: "*" , un mot de passe RDS en dur`). Ce fichier **ne fait pas partie de la landing zone déployable** — il existe uniquement pour donner aux scanners quelque chose de réaliste à détecter, afin que les chiffres “après” signifient quelque chose.
- **Après** : l’arborescence `terraform/` réelle.

Métrique	Avant	Après
tfsec CRITICAL	4	0
tfsec HIGH	11	0
tfsec checks réussis	5	70 (+ 20 ignores justifiés en ligne)
Checkov échoués	38	0
Checkov réussis	12	194
Scan de secrets Checkov	1 identifiant en dur détecté	—

La sortie brute des outils pour les deux exécutions est committée sous `docs/scan-results/{before,after}/` — non résumée ni retouchée à la main, donc les chiffres ci-dessus peuvent être re-vérifiés indépendamment en ouvrant ces fichiers ou en relançant les scanners localement (`tfsec terraform/` , `checkov -d terraform/`).

Le point à expliciter en entretien : 0 finding n’est pas la même chose que “0 risque accepté.” L’arborescence durcie compte encore 20 éléments ignorés par tfsec et 7 éléments sautés par Checkov — chacun est un finding réel correctement détecté par un scanner, revu individuellement, et jugé soit comme une limitation de la plateforme AWS (impossible à scoper davantage) soit comme un compromis délibéré et borné (ex. l’ `AdministratorAccess` du rôle break-glass). Le rôle du scanner est de forcer cette revue à avoir lieu, pas d’atteindre zéro en cachant des choses.

9. Runbook de déploiement (ce qu'appliquer réellement ceci impliquerait)

1. **Amorcer le backend Terraform** (en dehors du Terraform de ce dépôt, puisqu'un backend ne peut pas être créé par la config qui l'utilise) : un bucket S3 avec versioning et chiffrement activés, et une table DynamoDB pour le verrouillage du state.
 2. **Configurer** `terraform/environments/prod/backend.tf` avec les vrais noms de bucket/table, et `terraform.tfvars` (à partir du fichier `.example`) avec le véritable ID de compte, l'org/repo GitHub, et — élément critique — `human_admin_principal_arns` et `cluster_endpoint_public_access_cidrs` (si l'accès public est un jour activé).
 3. `terraform init && terraform plan && terraform apply` — crée le VPC, les clés KMS, les rôles IAM, et le cluster/node group EKS. Prévoir 15-20 minutes, majoritairement pour le provisionnement du control plane EKS.
 4. **Établir une connectivité privée** vers l'endpoint API EKS désormais privé : un VPN, une session SSM port-forwarding, ou un runner GitHub Actions auto-hébergé déployé dans le VPC (c'est une vraie décision opérationnelle qu'une équipe doit prendre, pas un détail à survoler).
 5. `aws eks update-kubeconfig`, puis `kubectl apply -f kubernetes/namespaces.yaml`, `kubernetes/rbac/`, `kubernetes/network-policies/` (la `GlobalNetworkPolicy` Calico nécessite précisément que les CRD de Calico soient déjà installées comme moteur de policy — via Helm, aux côtés du VPC CNI d'AWS qui continue de gérer l'attribution d'adresses IP).
 6. Installer l'External Secrets Operator via Helm, puis `kubectl apply -f kubernetes/external-secrets/` (après avoir remplacé l'ARN de rôle placeholder dans `service-accounts.yaml` par le véritable output Terraform).
 7. Pousser une charge de travail ; vérifier que les workflows `terraform-plan.yml` / `terraform-security-scan.yml` / `image-scan-trivy.yml` sont bien rattachés à la protection de branche comme checks requis.
-

10. Limites connues et ce qu'une mise en production ajouterait

Être transparent sur ces points est plus crédible que de prétendre que le dépôt est un produit fini :

- **L'egress des security groups EKS est 0.0.0.0/0**, filtré par le NAT mais pas restreint aux listes de préfixes IP des services AWS. Une mise en production avec une tolérance au risque plus faible ajouterait un proxy de filtrage d'egress ou scoperait l'egress des security groups aux listes de préfixes gérées par AWS (`com.amazonaws.<région>.s3`, etc.) plus des IP de registre spécifiques — délibérément hors périmètre ici, en tant que ligne de "durcissement supplémentaire" plutôt que tenté et fait à moitié.

- **Aucun WAF / aucune couche de protection DDoS** (Shield/WAF) devant l'ALB — le périmètre du cahier des charges était la landing zone et le cluster, pas la bordure applicative.
- **Aucun pipeline automatisé de patch des nœuds/AMI** — le node group utilise un launch template mais aucun workflow Karpenter/mise à niveau automatique de node group géré n'est rattaché.
- **Mono-région** — pas de stratégie de reprise d'activité cross-région.
- **Les statements en lecture seule/ `ManageVpcNetworking` du rôle de déploiement CI/CD utilisent `Resource: "*"`** là où le modèle IAM d'AWS l'exige (documenté au §5.3) — un environnement de production avec des exigences de conformité plus strictes (ex. périmètre PCI-DSS) pourrait en plus utiliser une permissions boundary ou une SCP au niveau AWS Organizations pour plafonner ce rôle indépendamment de ce que dit sa propre policy, comme seconde couche indépendante.

11. Questions d'entretien anticipées

“Pourquoi ne pas simplement utiliser une seule policy IAM `AdministratorAccess` pour la CI et s'appuyer sur la revue humaine de chaque PR ?” Parce que cela rend le rayon d'impact de chaque PR équivalent à une compromission complète du compte si le pipeline lui-même est un jour compromis (une attaque par confusion de dépendance sur une GitHub Action, un runner qui fuit, etc.) — l'objectif de scoper le rôle de déploiement est que *même si* le pipeline est compromis, la portée de l'attaquant reste bornée à ce que cette landing zone touche, et exclut spécifiquement l'auto-modification et la destruction de la piste d'audit.

“Pourquoi cinq clés KMS plutôt qu'une seule ?” Le rayon d'impact. Une policy de clé est le genre de chose qu'on assouplit sous la pression (“ajoute juste ce principal pour que le nouveau service puisse déchiffrer”) ; séparer par domaine de données fait qu'une telle erreur affecte une catégorie de données, pas tout ce qui est chiffré dans le compte.

“Tu as dit qu'EKS est privé par défaut — est-ce que ça ne rend pas le pipeline inutile ?” Non — Terraform parle à l'**API de control plane** d'EKS (`eks.<région>.amazonaws.com`), toujours accessible depuis les runners hébergés par GitHub indépendamment de la visibilité de l'API server propre au cluster. Seuls `kubectl` / `helm` ont besoin d'une accessibilité réseau au cluster, et c'est une exigence séparée, explicitement documentée (VPN/SSM/runner auto-hébergé).

“Que changerais-tu si ceci devait passer un audit PCI-DSS ou ISO 27001 ?” Ajouter une permissions boundary/SCP comme second plafond indépendant sur le rôle CI, restreindre l'égress des security groups aux listes de préfixes AWS plutôt qu'à `0.0.0.0/0` , étendre la rétention de logs au-delà de 400 jours selon l'exigence précise de la norme, et formaliser l'usage du rôle break-glass en un runbook de réponse à incident documenté avec revue post-usage obligatoire (le contrôle technique existe ; l'enveloppe processus autour n'existe pas encore).

“Comment as-tu validé tout ça sans le déployer ?” `terraform validate` détecte les erreurs de syntaxe/référence et les incompatibilités de type sur les quatre modules et l’environnement racine. `tfsec` / `checkov` sont des analyseurs statiques qui parsent le HCL directement — ils n’ont pas besoin d’un cluster déployé pour détecter “ce security group autorise 0.0.0.0/0 sur le port 22.” `kubeconform` valide les manifests Kubernetes par rapport aux schémas OpenAPI officiels. Ce qu’aucun de ces outils ne détecte, c’est le comportement à l’exécution (est-ce que la trust policy IRSA de l’External Secrets Operator fonctionne vraiment de bout en bout, est-ce que le routage du NAT Gateway se comporte comme prévu sous charge) — cet écart est réel, et c’est exactement pourquoi “validé, pas déployé” est l’affirmation exacte, pas “testé.”

12. Glossaire

- **IRSA** (IAM Roles for Service Accounts) : un mécanisme qui permet à un ServiceAccount Kubernetes précis d’assumer un rôle IAM précis via fédération OIDC, afin que des pods individuels obtiennent des permissions AWS finement scopées au lieu d’hériter du rôle IAM du nœud EC2.
- **Fédération OIDC** : échanger un token émis par un fournisseur d’identité externe (ici, l’émetteur de tokens propre à GitHub Actions) contre des credentials AWS temporaires via `sts:AssumeRoleWithWebIdentity`, sans qu’aucune clé d’accès AWS de longue durée soit impliquée.
- **NACL** (Network ACL) : un pare-feu sans état, au niveau subnet, dans AWS, évalué en plus de (et non à la place de) security groups.
- **Pod Security Admission / Pod Security Standards** : l’application, intégrée à Kubernetes (aucun contrôleur supplémentaire nécessaire), au niveau namespace, de règles de durcissement des pods, selon l’un des trois niveaux : `privileged`, `baseline`, `restricted`.
- **ABAC** (Attribute-Based Access Control, contrôle d’accès par attributs) : un modèle d’autorisation IAM où l’effet d’une policy dépend de tags sur la ressource/requête (`aws:ResourceTag/...`) plutôt que d’un ARN de ressource fixe — utilisé ici pour que les actions de gestion des clés KMS puissent être larges en *action* mais étroites en *effet*.
- **Accès break-glass** : un chemin d’accès à privilèges élevés délibérément rare, fortement journalisé, réservé aux urgences, par opposition à l’accès opérationnel du quotidien.